

Creating comprehensive study notes from PDF documents ✓

Length*: 39,838 tokens | Cost: 80,850 credits | Cached for: 51m

ificial Neural Network (Pa

PDF SE, University of Asia Pacific

You are an expert teacher and study-note creator.

Read the entire PDF carefully from the first page to the last page.

Create comprehensive study notes in very simple English so that a beginner can understand everything without reading the original PDF.

Requirements:

1. Do NOT skip any important topic, concept, definition, formula, diagram

Show more

■ Complete Study Notes: Artificial Neural Network (Part 2)

Based on: Backpropagation & Feed Forward Neural Networks — Dept. of CSE,
University of Asia Pacific

SECTION 1: Feed Forward Neural Network (FFNN)

Topic: What is a Feed Forward Neural Network?

Easy Explanation: A Feed Forward Neural Network is the most basic type of artificial neural network. Think of it like a one-way road — information travels only in one direction: from the input, through the middle (hidden) layers, and finally to the output. There are no loops, no going back.

Important Points:

- "Feed Forward" means data flows in one direction only — forward.
- There are three main parts: Input Layer → Hidden Layer(s) → Output Layer.
- Each layer is made of small units called **neurons**.

The Three Layers Explained:

Layer	What It Does
Input Layer	Receives the raw data (like numbers, features)
Hidden Layer(s)	Does the actual processing — applies weights and activation functions
Output Layer	Gives the final answer (prediction)

How It Works — Step by Step:

1. You give input data to the Input Layer.
2. Each neuron takes the inputs, multiplies them by weights, adds a bias, and passes the result through an activation function.
3. The result moves forward to the next layer.
4. This continues until the Output Layer gives the final prediction.
5. **No loops. No going backward during prediction.**

SECTION 2: The Mathematics of a Single Neuron

Topic: How Does One Neuron Calculate Its Output?

Easy Explanation: Every neuron is like a tiny calculator. It takes all its inputs, multiplies each by a weight (importance value), adds them all up, adds a bias (an extra adjustment number), and then applies an activation function to decide the final output.

The Formula:

$$y = \phi(\sum_i w_i x_i + b)$$

What Each Symbol Means:

Symbol	Name	Meaning
x_i	Input features	The data going into the neuron ($x_1, x_2, x_3, \dots$)
w_i	Weights	How important each input is
b	Bias	A constant added to shift the result (like a fine-tuner)
Σ	Summation	Add everything together

Symbol	Name	Meaning
ϕ (phi)	Activation function	A function that decides the final output
y	Output	The result the neuron produces

Step-by-Step Meaning:

1. Take each input (x_1, x_2, x_3)
2. Multiply each by its weight ($w_1x_1 + w_2x_2 + w_3x_3$)
3. Add the bias $b \rightarrow$ this gives z (called the **transfer function** or **pre-activation value**)
4. Pass z through the activation function $\phi \rightarrow$ this gives the output y

Diagram Description (Page 4): The diagram shows a single neuron with:

- Three inputs (x_1, x_2, x_3) on the left
- Each connected by a weight (w_1, w_2, w_3)
- A summation box (Σ) that adds weighted inputs + bias b
- The result z goes into the activation function box (ϕ)
- The final output y comes out on the right

Why This Diagram is Important: It visually shows the inside of any single neuron — the building block of all neural networks.

SECTION 3: One Hidden Layer (Shallow Neural Network)

Topic: What Happens When Multiple Neurons Work Together?

Easy Explanation: A "shallow" neural network has just one hidden layer, but that layer can contain many neurons working at the same time. Instead of one neuron, you have a whole row of neurons processing your input simultaneously.

Key Terms for a Layer:

Symbol	Meaning
$x \in \mathbb{R}^n$	Input vector — n numbers going in
m	Number of neurons in the hidden layer
$W \in \mathbb{R}^{m \times n}$	Weight matrix — a table of all weights

Symbol	Meaning
$\mathbf{b} \in \mathbb{R}^m$	Bias vector — one bias per neuron
ϕ	Activation function (applied to each neuron separately)

Formula for Each Neuron:

$$z_i = \sum w_i x_i + b$$

Combined Formula for the Whole Layer (Matrix Form):

$$\mathbf{z} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

This means: instead of calculating one neuron at a time, we do all neurons at once using matrix multiplication.

Full Output Formula for a Shallow Network:

$$y = g(\mathbf{W}_{12} \cdot f(\mathbf{W}_{11} \cdot \mathbf{X}_{11} + \mathbf{b}_{11}) + \mathbf{b}_{12})$$

Breaking This Down:

- $\mathbf{W}_{11}, \mathbf{b}_{11} \rightarrow$ weights and biases of the **hidden layer**
- $f(\cdot) \rightarrow$ activation function of the hidden layer
- $\mathbf{W}_{12}, \mathbf{b}_{12} \rightarrow$ weights and biases of the **output layer**
- $g(\cdot) \rightarrow$ activation function of the output layer

Diagram Description (Page 6): The diagram shows:

- Two inputs: x_1 and x_2 on the left
- Two hidden layer neurons (each with their own weights and biases: $w_{111}, w_{112}, w_{121}, w_{122}$ and biases b_{11}, b_{12})
- The outputs of the hidden neurons (w_{211} and w_{221}) go into a final summation
- A bias b is added
- Then an activation function ϕ gives the final output y

Why This Diagram Matters: It shows how a hidden layer works — multiple neurons process the same input in parallel, and their outputs are combined.

SECTION 4: Multiple Hidden Layers (Deep Neural Network)

Topic: What is a Deep Neural Network?

The General Formula:

$$y = g(W_L \cdot f_L(W_{L-1} \cdot \dots f_2(W_2 \cdot f_1(W_1 \cdot X + b_1) + b_2) \dots + b_{L-1}) + b_L)$$

Simple Meaning:

- Start with input X
- Apply layer 1: multiply by W_1 , add b_1 , apply $f_1()$
- Feed result into layer 2: multiply by W_2 , add b_2 , apply $f_2()$
- Continue for all L layers
- The final layer uses $g()$ to produce the output y

Diagram Description (Page 7): The diagram uses colored brackets (green, yellow, blue) to show how the formula nests — each bracket represents one layer wrapping around the previous one. This nested structure is why it's called "deep."

Key Point: The deeper the network, the more complex features it can learn — but it also becomes harder to train.

SECTION 5: Multiple Output Neurons

Topic: How Do We Get Output When There Are Many Output Neurons?

Easy Explanation: Sometimes a network needs to classify into more than 2 categories (e.g., cat, dog, or bird). In that case, the output layer has multiple neurons — one per class.

Three Main Approaches:

Approach 1 — Sigmoid + Threshold (for binary per class):

- Each output neuron uses a **sigmoid** activation function $\rightarrow$ gives a value between 0 and 1
- Apply a threshold (usually 0.5): if output $> 0.5 \rightarrow$ class is 1, else $\rightarrow$ class is 0
- Good for: multi-label problems (where multiple classes can be true at once)

Approach 2 — Softmax + Argmax (most common for classification):

- **Softmax** converts all output neuron values into probabilities that add up to 1
- **Argmax** picks the neuron with the highest probability as the predicted class
- Good for: single-label classification (only one class is correct)

Approach 3 — Custom Binary Output:

- Add a final layer on top
- Apply a logic rule (like majority vote) or a learned combination to get a binary result

SECTION 6: Backpropagation

Topic: What is Backpropagation and How Does It Work?

Easy Explanation: After a neural network makes a prediction, it compares that prediction to the correct answer. If it's wrong, it needs to fix itself. Backpropagation is the process of going **backward** through the network to figure out how much each weight contributed to the error, and then adjusting those weights to do better next time.

Think of it like a teacher marking an exam and telling each student exactly where they went wrong.

The 5 Steps of Backpropagation:

Step 1 — Forward Pass

- Send the input through all layers normally
- Calculate the output at each layer
- Store all intermediate values (you'll need them later)

Step 2 — Compute Output Error

$$\delta^L = (z^L - y) \times \sigma'(a^L)$$

Symbol	Meaning
δ^L	Error at the output layer (called "delta")
z^L	The network's predicted output
y	The true/correct answer
$\sigma'(a^L)$	Derivative of the activation function at the output layer

Simple Meaning: Find the difference between what the network predicted and what the correct answer was, then adjust it using the derivative of the activation function.

Step 3 — Backward Pass (for each hidden layer, from last to first)

$$\delta^l = (w^{l+1})^T \cdot \delta^{l+1} \times \sigma'(a^l)$$

Symbol	Meaning
δ^l	Error at layer l
$w^{l+1}{}^T$	Transpose of the weight matrix of the next layer
δ^{l+1}	Error from the next layer (already calculated)
$\sigma'(a^l)$	Derivative of activation function at layer l

Simple Meaning: Carry the error backward, layer by layer, using the weights and derivatives. This tells each layer "how much" it contributed to the mistake.

Step 4 — Compute Gradients

For weights:

$$\partial L / \partial w^l = \delta^l \cdot (z^{l-1})^T$$

For biases:

$$\partial L / \partial b^l = \delta^l$$

Simple Meaning:

- The gradient for weights = how much the loss changes when that weight changes
 - The gradient for biases = the delta itself
 - These gradients tell us in which direction to adjust the weights
-

Step 5 — Update Weights and Biases

$$w^l = w^l - \alpha \times (\partial L / \partial w^l) \quad b^l = b^l - \alpha \times (\partial L / \partial b^l)$$

Symbol	Meaning
w^1	Updated weight
α (alpha)	Learning rate — how big a step to take
$\partial L / \partial w^1$	Gradient (direction of error increase)

Simple Meaning: Move the weights a little bit in the direction that reduces the error. The learning rate α controls how big the step is — too large and you overshoot, too small and learning is very slow.

SECTION 7: Loss Functions

Topic: How Do We Measure How Wrong the Network Is?

Easy Explanation: A Loss Function measures the difference between what the network predicted and the correct answer. The goal of training is to minimize (reduce) this loss value to zero.

Four Loss Functions Covered:

1. Binary Cross-Entropy Loss

$$L = -[y \cdot \log(\hat{p}) + (1-y) \cdot \log(1-\hat{p})]$$

Symbol	Meaning
y	True label (0 or 1)
$\hat{p}$	Predicted probability
$\log$	Natural logarithm

When to Use: Binary classification (only 2 classes — Yes/No, 0/1)

Simple Meaning: Penalizes the network heavily when it's very confident but very wrong.

2. Categorical Cross-Entropy Loss

$$L = -\sum_j y_j \cdot \log(\hat{p}_j)$$

Symbol	Meaning
y_j	True label for class j (1 for correct class, 0 for others)
$\hat{p}_j$	Predicted probability for class j
C	Total number of classes

When to Use: Multi-class classification (3 or more classes)

3. Mean Squared Error (MSE)

$$L = (1/n) \sum (y - \hat{y})^2$$

When to Use: Regression tasks (predicting continuous numbers, like house prices)

Simple Meaning: Averages the squared differences between predicted and actual values.

4. Mean Absolute Error (MAE)

$$L = (1/n) \sum |y - \hat{y}|$$

When to Use: Regression tasks — less sensitive to outliers than MSE.

SECTION 8: Gradient Descent in Backpropagation

Topic: What is Gradient Descent?

Easy Explanation: Imagine you're standing on a hill and want to get to the lowest point (the valley). You look around, find the steepest downhill direction, take a step that way, and repeat. That's gradient descent — the network keeps adjusting weights in the direction that reduces loss.

Diagram Description (Page 11): The diagram shows:

- **Y-axis:** $J(w)$ — the loss/cost function value
- **X-axis:** w — the weight value
- A U-shaped curve (like a bowl)
- A black dot labeled "Initial weight" — where training starts (high on the curve)

- Arrows pointing downward and to the left, showing each update step
- At the bottom: "Global cost minimum $J_{\min}(w)$ " — the ideal target

Why This Diagram Matters: It perfectly illustrates the goal — start from some random weight, use gradients to find the direction of steepest descent, and keep updating until you reach the minimum loss.

Key Insight: The gradient tells you the slope at your current position. Moving opposite to the gradient goes downhill (reduces loss).

QUICK REVISION NOTES

- **FFNN** = data flows forward only, no loops
- **Neuron formula:** $y = \phi(\sum w_i x_i + b)$ — weighted sum + bias $\rightarrow$ activation
- **Shallow NN** = 1 hidden layer; **Deep NN** = multiple hidden layers
- **Deep NN formula:** Nested application of weight matrices and activation functions
- **Backpropagation has 5 steps:** Forward Pass $\rightarrow$ Compute Error $\rightarrow$ Backward Pass $\rightarrow$ Compute Gradients $\rightarrow$ Update Weights
- δ^L = output error; δ^l = hidden layer error (propagated backward)
- **Learning rate α** controls step size in weight updates
- **Loss functions:** Binary Cross-Entropy (2 classes), Categorical Cross-Entropy (many classes), MSE and MAE (regression)
- **Gradient Descent** = minimize loss by moving weights opposite to the gradient

MOST IMPORTANT EXAM TOPICS

1. The formula for a single neuron and what each variable means
2. The difference between Shallow NN and Deep NN
3. All 5 steps of Backpropagation with formulas
4. What a loss function is and when to use each type
5. What gradient descent is and how it connects to backpropagation
6. What a learning rate is and why it matters
7. The weight update rule: $w = w - \alpha \times \partial L / \partial w$
8. Softmax vs Sigmoid for output layer
9. What δ (delta) represents in backpropagation
10. Why we need an activation function

VIVA QUESTIONS AND ANSWERS

Q1: What is a Feed Forward Neural Network? A: A neural network where data flows in only one direction — from input to output — with no loops or feedback.

Q2: What is the role of weights in a neural network? A: Weights determine how important each input is. Larger weight = more influence on the output.

Q3: What is a bias in a neural network? A: A bias is an extra value added to the weighted sum, allowing the activation function to shift — it helps the model fit the data better.

Q4: What is an activation function and why is it needed? A: An activation function introduces non-linearity into the network. Without it, the network can only learn linear relationships, which is too simple for most real problems.

Q5: What is backpropagation? A: It is the algorithm that calculates how much each weight contributed to the prediction error and adjusts the weights to reduce that error.

Q6: What is the learning rate? A: The learning rate (α) controls how big a step the network takes when updating weights. Too high = unstable training; too low = very slow training.

Q7: What is the difference between MSE and Cross-Entropy loss? A: MSE is used for regression (predicting numbers). Cross-Entropy is used for classification (predicting categories).

Q8: What does the gradient tell us? A: The gradient tells us the direction and steepness of increase in the loss function. We move opposite to it to reduce the loss.

Q9: What is the difference between Softmax and Sigmoid? A: Sigmoid gives independent probabilities per neuron (for multi-label). Softmax gives probabilities that sum to 1 (for single-label multi-class).

Q10: Why is gradient descent needed in backpropagation? A: Because we need to minimize the loss function, and gradient descent is the method that moves the weights step by step toward the minimum.

GLOSSARY OF TERMS

Term	Simple Definition
Neuron	A basic computing unit in a neural network that takes inputs, processes them, and produces an output

Term	Simple Definition
Weight (w)	A number that shows how important each input is to a neuron
Bias (b)	An extra number added to help the model adjust its output independently of input
Activation Function (ϕ)	A mathematical function applied to a neuron's output to introduce non-linearity
Feed Forward	A network where data only moves forward, never backward
Hidden Layer	Layer(s) between the input and output that do the heavy processing
Shallow NN	A neural network with only one hidden layer
Deep NN	A neural network with multiple hidden layers
Backpropagation	The process of going backward through the network to compute gradients and update weights
Gradient	The slope/direction of the loss function — tells us how to adjust weights
Gradient Descent	An optimization method that moves weights in the direction that reduces loss
Learning Rate (α)	Controls how big each weight update step is
Loss Function	A measure of how wrong the network's prediction is
MSE	Mean Squared Error — average of squared differences between predicted and actual values
MAE	Mean Absolute Error — average of absolute differences
Cross-Entropy	A loss function used for classification problems
Sigmoid	An activation function that outputs values between 0 and 1
Softmax	An activation function that converts outputs into probabilities summing to 1
Argmax	Picks the index with the highest value — used to select the predicted class
Delta (δ)	The error signal at each layer during backpropagation
Forward Pass	Running input data through the network to get a prediction

Term	Simple Definition
Backward Pass	Sending the error signal backward through the network
Global Minimum	The point where the loss is at its absolute lowest — the ideal training target
Transfer Function (z)	The weighted sum before the activation function: $z = Wx + b$

💡 **Study Tip:** Master the 5 steps of backpropagation with their formulas — this is the most exam-critical topic in the entire PDF. Draw the diagrams by hand at least once to understand the flow visually.

